Guia de estudo — Sunnyland (projeto atual)

Material didático para o grupo estudar o código, compreender como as cenas se conectam e preparar uma apresentação. O foco é explicar o que existe na cópia atual do projeto, não atribuir autoria nem apontar culpados.

> *Escopo desta edição: Godot 4.7.1, projeto em* `Sunnyland_Trab_final`*. O guia descreve a cópia local que foi analisada depois da integração do mapa, dos inimigos e dos itens enviados para a fase 4. Naquele momento, essa integração da fase 4 ainda estava como alteração local não commitada. O teste headless confirmou que as cenas 1–5 carregam; não substitui jogar cada fase.*

1. Como estudar este guia

1.. Leia primeiro a visão geral e o fluxo de inicialização.

2.. Abra no Godot `principal.tscn` e acompanhe o fluxo para o seletor e para uma fase.

3.. Compare as duas famílias de jogador antes de estudar os inimigos: elas explicam várias diferenças entre as fases.

4.. Escolha um objeto (por exemplo, uma gema) e siga este caminho: cena `.tscn` → script `.gd` → sinal/grupo → HUD ou jogador.

5.. Termine com as perguntas de revisão e a pauta de apresentação. Se o grupo quiser confirmar comportamento, teste no jogo aquilo que o guia identifica como observação estática.

Vocabulário essencial do Godot

Termo	Explicação simples	Exemplo neste projeto
Nó (Node)	Peça de comportamento ou organização da árvore do jogo.	`CharacterBody2D`, `Area2D`, `Camera2D`.
Cena (Scene)	Conjunto reutilizável de nós guardado em `.tscn`; pode ser instanciado.	`actors/player.tscn`, `prefabs/escada.tscn`.
Instância	Cópia de uma cena dentro de outra cena.	Cada águia colocada no mapa é uma instância de `actors/eagle.tscn`.
Script	Código GDScript ligado a um nó para lhe dar comportamento.	`scrips/player.gd` controla o jogador moderno.
Sinal (signal)	Mensagem emitida por um nó e recebida por outro, sem exigir que os dois sejam o mesmo objeto.	Um inimigo sinaliza dano; o HUD atualiza as vidas.
Grupo (group)	Etiqueta que permite localizar vários nós por função.	`jogador`, `player`, `inimigos`, `diamantes`.
Recurso (resource)	Dado reutilizável que uma cena/script pode carregar.	`Texture2D`, `PackedScene`, `TileSet`, `SpriteFrames`.
Autoload	Nó/serviço global, disponível durante toda a execução.	`controladorFases`.
TileMap / TileSet	Ferramentas para montar mapas usando peças de uma imagem e associar colisões.	Terreno de `fases/1/fase.tscn` a `fases/5/fase.tscn`.

2. O que o jogo faz

Sunnyland é um jogo de plataforma 2D. O jogador percorre fases, evita ou derrota inimigos, coleta itens e tenta chegar ao fim da fase. O projeto foi construído e ampliado em etapas, por isso há uma implementação mais antiga e outra mais recente para alguns elementos (especialmente jogador, inimigos, coletáveis e HUD).

Mapa mental da arquitetura

text

```
project.godot
└── principal.tscn                        cena de entrada
    ├── Menu                               menu inicial
    └── nó que recebe a fase atual
        ├── fases/0/fase.tscn              seletor
        └── fases/1..5/fase.tscn           níveis jogáveis
            ├── TileMap / plataformas
            ├── jogador legado ou moderno
            ├── inimigos e itens
            ├── barreiras / áreas de interação
            └── câmera e HUD

Autoload:
└── controladorFases                       carrega fases e mantém dados
```

Fluxo de inicialização e navegação

1.. `project.godot` inicia `principal.tscn`. O projeto define uma resolução lógica de 320×180 e uma janela de 960×540, com expansão de conteúdo adequada a `canvas_items`.

2.. `scripts/principal.gd` prepara o controlador global. Ele fornece o nó onde as cenas de fase serão adicionadas e os dados de fase.

3.. O menu oferece Jogar e Sair. Jogar solicita o id `0`; Sair termina o aplicativo.

4.. A fase `0` é o seletor, não um nível jogável. Os botões do seletor solicitam ids de fase `1` a `5`.

5.. `scripts/controladorFases.gd` resolve o caminho `res://fases/<id>/fase.tscn`, carrega a cena, remove a anterior e instancia a nova.

6.. A cena de fase usa `scripts/fase.gd` para associar seu id a um recurso `dadosFase`.

Importante para explicar: existem dados de desbloqueio/conclusão, mas a leitura estática encontrou um descompasso: o seletor oferece chamadas diretas de carregamento e o carregador não barra ids bloqueados. Assim, os dados não equivalem, por si sós, a uma trava efetiva de acesso.

3. Configuração e recursos do projeto

Configurações principais

- Engine: Godot 4.7.1.
- Cena principal: `principal.tscn`.
- Autoload identificado: `controladorFases`.
- Ações de entrada: `pulo`, `irEsquerda`, `irDireita` e `baixo`; também há aliases como `jump`, `left`, `right`, `up` e `down` usados por algumas implementações.
- Teclado e controle: as ações principais incluem Espaço para pular, A/D e setas para mover, e S/seta para baixo; algumas ações têm mapeamentos para controle. Consulte a seção `[input]` em `project.godot` para o mapa completo do computador usado na apresentação.
- Camadas de física: estático, dinâmico, perigo, itens, hitbox e hurtbox. Camadas e máscaras definem com que objetos cada colisão interage.

Recursos utilizados e o papel de cada um

Recurso	Onde aparece	Para que serve
GDScript (`.gd`)	`scripts/`, `scrips/`, `prefab/`	Regras de jogo, interface, movimento e respostas a colisões.
Cenas Godot (`.tscn`)	`fases/`, `actors/`, `prefab/`, `prefabs/`	Montam níveis, jogadores, inimigos, interfaces e objetos reutilizáveis.
TileSet/TileMap	Cenas de fase	Montam o terreno e armazenam colisão de cada tile.
Imagens/spritesheets	`assets/`	Aparência de personagens, inimigos, fundo, itens e interface.
`SpriteFrames`/animações	Cenas de atores	Define animações como parado, corrida, salto, dano e morte.
`AudioStream`	`assets/musica/` e cenas	Música e sons usados pelo projeto.
`Area2D` e `CollisionShape2D`	Escadas, hitboxes, itens e saídas	Detectam entrada, saída e contato sem necessariamente bloquear movimento.
`CharacterBody2D`	Jogadores e inimigos	Movimento controlado por física e colisões.
`CanvasLayer`, `Control`, `Label`, `TextureRect`	HUD e menus	Elementos de interface que permanecem sobre o cenário.
`Tween`/temporizadores	Dano, coleta e transições	Animações temporárias, piscar, fade e atrasos.
Arquivos `.import` e cache `.godot/`	Importação de assets/editor	Metadados e resultados gerados pelo Godot; não são a arte-fonte do jogo.
`.ase` / `.xcf`	Alguns diretórios de arte	Fontes editáveis de imagens, quando presentes.

Pastas e grafias que o grupo vai encontrar

- `actors/`: cenas mais novas de jogador e inimigos.
- `fases/`: seletor e cenas dos níveis 1–5.
- `prefab/`: prefabs antigos e recursos como jogador, barreiras e coletáveis.
- `prefabs/`: prefabs mais recentes de itens, escada, saída, plataforma e HUD.
- `scripts/`: scripts gerais, controlador e compatibilidade.
- `scrips/`: nome histórico usado para scripts dos sistemas mais novos

(incluindo jogador, combate, inimigos e itens). A grafia é diferente, mas a pasta está referenciada por caminhos `res://scrips/...`.

- `assets/`: arte, fundos, sprites, música e demais arquivos de conteúdo.

Ao abrir um caminho, respeite a grafia e o singular/plural exatos. `prefab` e `prefabs`, assim como `scripts` e `scrips`, são diretórios diferentes.

4. As duas implementações de jogador

As fases não usam todas a mesma cena de jogador. Isso é uma escolha histórica da evolução do projeto e uma razão importante para alguns comportamentos diferentes.

Característica	Legado	Mais recente
Cena/script	`prefab/jogador/jogador.tscn` e `prefab/jogador/jogador.gd`	`actors/player.tscn` e `scrips/player.gd`
Fases que o usam	1, 2 e 4	3 e 5
Movimento	Gravidade, direção, pulo variável, coyote time e descida de plataforma.	Estados de movimento, pulo, queda, dano, morte e escalada; inclui suporte a aliases de entrada.

Característica	Legado	Mais recente
Combate	Funções de vida e rebote, com conexões mais ligadas à cena.	`HitBox`/`HurtBox`, dano e knockback por sinais.
Escalada	Não possui o mesmo sistema de escalada.	Estado `CLIMB` e comunicação com a área da escada.
Compatibilidade	Nome histórico `jogador`.	Grupo/classe `Player`; a cena também registra grupos para compatibilidade.
Cura	A implementação legada soma vida sem o mesmo limite observado no jogador moderno.	A implementação moderna limita a cura à vida máxima.

O nome `JogadorCompat` centraliza reconhecimento do personagem por tipo, grupo ou nome. É uma camada de compatibilidade; não transforma os dois jogadores em uma única implementação.

Ciclo de um jogador moderno

1.. A cena cria o corpo do jogador, sprite animado, colisões, hurtbox e hitbox.

2.. `_ready()` registra grupos, localiza a câmera quando aplicável e prepara referências/sinais.

3.. `_physics_process()` lê entradas, calcula gravidade e movimento e chama a física do corpo.

4.. O estado atual escolhe a animação e limita quais ações são válidas (por exemplo, dano ou morte interrompem movimento normal).

5.. Ao entrar numa área de escada, a escada chama `entrar_na_escada`; ao sair, chama `sair_da_escada`. O jogador decide se muda para o estado de escalada.

6.. Ao receber dano, ajusta vida/knockback, toca animação e pode entrar no estado de ferido ou morto.

Combate: hitbox e hurtbox

text

```
HitBox detecta HurtBox
        │
        ├── impede autocolisão / fogo amigo de inimigo
        └── emite hit_registered (alvo, dano, força)
                    │
                    ▼
HurtBox chama take_damage(...)
        │
        └── emite damage_received
                    │
                    ▼
Jogador ou inimigo altera vida, movimento e estado
```

`HitBox` e `HurtBox` são `Area2D`, não são a mesma coisa que o corpo físico do personagem. As camadas 5 e 6 foram nomeadas para hitbox e hurtbox; as máscaras determinam se elas se detectam. A lógica de combate usa sinais para que o detector não precise conhecer todos os detalhes internos do alvo.

5. Mecânicas principais

Movimento, pulo e dano

• O chão e as plataformas fornecem suporte físico; o jogador aplica gravidade quando não está no chão.

• O pulo variável permite modular a altura de acordo com o tempo de pressão da entrada, quando previsto pela implementação do jogador.

• Coyote time é uma pequena janela depois de sair de uma borda em que o pulo ainda pode ser iniciado.

• O jogador mais recente tem estados explícitos (`IDLE`, `RUN`, `JUMP`, `FALL`, `HURT`, `DIE`, `CLIMB`).

• O sistema moderno também verifica tiles com dado customizado de perigo, além das colisões de inimigos.

• A queda para fora do mapa é tratada pela cena de barreiras/chão, que pode mostrar a tela de morte e solicitar reset.

Inimigos

• Os atores mais novos são gambá/opossum, sapo e águia. Eles têm movimentos próprios e respostas a dano/morte; a configuração e a cena definem velocidade, direção, patrulha e outros valores.

• O gambá antigo e o sapo antigo permanecem em `prefab/inimigos/`.

• A águia antiga `aguia.gd` é um caminho separado da águia moderna `scrips/eagle.gd`.

• O grupo `inimigos` é usado para evitar interações de combate indesejadas entre inimigos.

• Em alguns sistemas, o jogador pode derrotar um inimigo ao aterrissar sobre ele; não presuma que isso esteja padronizado em todas as versões.

Coletáveis, vida e pontuação

• A cereja restaura vida quando reconhece o jogador.

• A gema é um coletável contável; o HUD de pontuação localiza gemas/grupos e atualiza o contador.

• Existem famílias antiga e moderna de cereja/gema. Algumas usam `JogadorCompat`; outras têm sinal/feedback próprios.

• A cena de gema base remove o item ao coletar; as subclasses filtram o corpo para aceitar o jogador.

• O HUD de vidas observa vida, dano e recuperação. A versão moderna e a antiga não se conectam e não se comportam da mesma maneira em todas as fases.

Escadas, saída e áreas secretas

• `prefabs/escada.tscn` combina uma imagem de escada com uma área não sólida e `scrips/escada.gd`.

• A escada detecta um corpo e chama métodos do jogador, sem codificar o movimento de escalada dentro da própria escada.

• A fase 5 já tinha escadas; foram adicionados dois segmentos à fase 3 na cópia local, sem alterar a lógica compartilhada.

• `prefabs/porta_saida.tscn` apresenta interação/prompt e transição de vitória.

• As duas áreas secretas têm responsabilidades distintas: uma altera o limite da câmera e a outra anima a visibilidade de um polígono.

6. Guia de cada fase

Id	Cena	O que estudar na árvore	Sistemas e observações
0	`fases/0/fase.tscn`	Menu de seleção e botões para ids 1–5.	Não é uma fase de plataforma. Os botões chamam o controlador.
1	`fases/1/fase.tscn`	TileMap, jogador legado, inimigo gambá, cereja, barreiras e plataformas direcionais.	Padrão antigo; sinais de dano/recuperação são conectados na cena.
2	`fases/2/fase.tscn`	TileMap, jogador legado e barreiras.	Cena estruturalmente mais simples que as fases 3 e 5. Verifique a própria cena no editor para os elementos presentes.
3	`fases/3/fase.tscn`	Jogador moderno, sapos, gambás, águias, gemas/cerejas, câmera e grupo `Escadas`.	Usa estados e combate modernos. Há duas instâncias de UI de vida e duas de pontuação na cena, um ponto a conferir visualmente. Duas instâncias de escada foram adicionadas à cópia local.
4	`fases/4/fase.tscn`	Jogador legado, TileMap atualizado, nós `Inimigos/Item` e instâncias adicionais.	A integração do ZIP deixa três águias, dois sapos, dois gambás, três gemas e uma cereja no mapa. Mistura jogador legado e atores mais novos.
5	`fases/5/fase.tscn`	Jogador moderno, câmera, HUD, oito escadas, duas áreas secretas, saída, inimigos e coletáveis.	A fase mais completa em sistemas modernos; bom exemplo para estudar escalada, saída e câmera.

Roteiro de inspeção de uma fase no editor

1.. Abra `fases/<id>/fase.tscn`.
2.. Expanda a árvore e localize `TileMap`, jogador, nós de inimigos, itens e UI.
3.. Selecione o jogador e confira o script anexado e o nome dos grupos.
4.. Selecione cada instância de cena e use Abrir Cena para ver seus nós

internos.

5.. No inspetor, confira camadas/máscaras de colisão e sinais conectados.
6.. Execute a fase e teste movimentos, coleta, dano e finalização. O carregamento

sem erro não prova que todas as conexões ou controles são corretos.

7. Referência dos scripts, por área

Os caminhos abaixo são relativos à raiz do projeto. Arquivos com grafias parecidas podem pertencer a sistemas diferentes: por exemplo, `scrips/player.gd` e `prefab/jogador/jogador.gd`.

Entrada, fases e compatibilidade

Script	Responsabilidade	Onde procurar / por que importa
`scripts/principal.gd`	Conecta a cena principal ao controlador e prepara sinais de inimigos.	Inicialização. O `_ready()` corre no começo, antes de níveis carregados depois.
`scripts/controladorFases.gd`	Carrega/remove cenas, consulta dados, registra conclusão/desbloqueio.	Autoload em `project.godot`; núcleo da navegação.
`scripts/dadosFase.gd`	Recurso com id, nome, concluída e desbloqueada.	Instâncias embutidas em `principal.tscn`.
`scripts/fase.gd`	Liga o id da raiz de uma cena de fase aos dados correspondentes.	Usado nas cenas `fases/0` a `fases/5`.
`scripts/menu.gd`	Responde aos botões Jogar e Sair e alterna o menu.	Anexado ao menu de `principal.tscn`.
`fases/0/seletor.gd`	Callbacks dos botões do seletor.	Chama `carregarFase(id)`.
`scripts/jogador_compat.gd`	Reconhece jogadores por classe, grupo ou nome.	Compartilhado por itens, escadas, porta e áreas; evita depender de um único nome.
`fases/1/fase.tscn.gd`	Guarda id e referência de início de jogador para uma fase.	Não foi encontrada referência de cena para esse script no fluxo analisado.
`scripts/fundo.gd`	Move camadas de parallax para dar sensação de profundidade.	`prefab/controle/fundo.tscn`.
`scripts/inimigo.gd`	Base antiga mínima para inimigo (`Node`).	Herdada por `aguia.gd`; não é a base dos inimigos modernos.

Jogadores, hitboxes e interface

Script	Responsabilidade
`prefab/jogador/jogador.gd`	Movimento legado, gravidade, pulo variável, coyote time, plataformas, vida, rebote, morte e reset.
`prefab/jogador/vida.gd`	HUD legado de vida e callbacks de dano/recuperação.
`scrips/player.gd`	Jogador moderno: estados, movimento, entrada, dano, vida, morte, escadas e detecção de tiles perigosos.
`scrips/hit_box.gd`	Detecta hurtboxes válidas, calcula/intermedeia dano e knockback e emite registro do acerto.
`scrips/hurt_box.gd`	Recebe uma chamada de dano e emite `damage_received`.
`scrips/vida.gd`	HUD moderno de vida; procura jogador e conecta sinais de dano/recuperação.
`scrips/pontuacao_ui.gd`	Conta gemas/coletáveis e atualiza o contador.

Itens e interações

Script	Responsabilidade
`scrips/item_base.gd`	Base moderna para animação/remoção após coleta.
`scrips/cherry.gd`	Cereja moderna; valida jogador e sinaliza recuperação.
`scrips/gem.gd`	Gema moderna; grupos de coletável/diamante e sinal de coleta.
`prefab/coletaveis/cereja/cereja.gd`	Cereja antiga com feedback e recuperação de vida.
`prefab/coletaveis/gema/gema.gd`	Gema antiga com feedback visual.
`scrips/escada.gd`	Área detectável que avisa entrada e saída do jogador.
`scrips/porta_saida.gd`	Prompt, interação, fade e tela de vitória da saída.
`scrips/moving_plataform.gd`	Movimento de plataforma com direção, distância e espera.
`scrips/area_secreta_01.gd`	Anima o limite esquerdo da câmera ao entrar/sair de uma região.
`scrips/area_secreta_02.gd`	Anima a opacidade/visibilidade do polígono da área.

Inimigos, barreiras e encerramento

Script	Responsabilidade
`scrips/opossum.gd`	Inimigo moderno: patrulha, dano, estados, animação e morte.
`scrips/frog.gd`	Inimigo moderno que salta e responde a colisões/dano.
`scrips/eagle.gd`	Inimigo moderno voador, com movimento, dano e estados.
`prefab/inimigos/gamba/gamba.gd`	Gambá antigo; inclui interação de dano e derrota por pisão.
`prefab/inimigos/sapo/sapo.gd`	Sapo antigo; comportamento de salto e mudança de direção.
`aguia.gd`	Águia antiga simples, relacionada a `prefab/inimigos/gaviao/aguia.tscn`.
`prefab/controle/chao.gd`	Resposta a queda para fora do mapa e chamada de morte/reset.
`prefab/controle/final.gd`	Finalização antiga e retorno/atualização do seletor.
`prefab/controle/mensagens.gd`	Mostra/esconde mensagens de vitória e morte.

8. Leitura de sinais: exemplo didático

Um sinal é como uma notificação com nome e dados. Exemplo conceitual de dano:

1.. O corpo encosta em uma `HitBox`.
2.. A hitbox verifica se a área oposta é uma `HurtBox` válida.
3.. A hurtbox recebe `take_damage(...)` e emite `damage_received`.
4.. O script do personagem escuta o sinal e decide reduzir vida, aplicar

knockback e tocar uma animação.

5.. O HUD escuta a mudança de vida e redesenha os ícones.

Isso separa detecção (áreas de colisão), regra (script do personagem) e apresentação (HUD). Ao depurar, confira cada elo; encontrar uma função não prova que o sinal esteja conectado à função.

9. Pontos de atenção para compreender e testar

Estes pontos são bons tópicos de estudo; não são uma avaliação de quem fez cada parte.

1.. Duas famílias de jogador: as fases 1, 2 e 4 usam o legado; 3 e 5 usam o

moderno. Comparar cenas e sinais explica por que nem toda animação ou mecânica aparece em todas elas.

2.. Compatibilidade da águia: `actors/eagle.tscn` serializa uma conexão de

sinal para `_on_hit_box_area_entered`, mas o script moderno `scrips/eagle.gd` não define esse método; o script conecta `hit_registered` no `_ready()`. É uma conexão antiga que vale conferir no editor e durante execução.

3.. Dano na fase 4: ela mistura jogador legado com inimigos modernos. A

leitura das conexões não mostra claramente como os sinais desses inimigos chegam ao HUD legado. Teste o dano dentro da fase.

4.. Desbloqueio de fases: existem valores de conclusão/desbloqueio, mas as

chamadas de carregamento do seletor não demonstram uma trava efetiva.

5.. Saída: a rotina em `scrips/porta_saida.gd` usa explicitamente o id 5 ao

marcar conclusão/desbloqueio; não é uma regra geral para qualquer fase.

6.. Player compatível e tipagem da porta: reconhecer um nó como jogador não

garante que ele possa ser tratado como a classe concreta `Player`; testar uma porta com o jogador legado é um caso de borda importante.

7.. HUD duplicado na fase 3: a cena contém duas instâncias de HUD de vida e

duas de pontuação; confirmar visualmente se isso é intencional.

8.. Rebote e colisão: o jogador moderno pode rebater ao receber contato de

hurtbox sem que o callback citado filtre ali se foi um pisão por cima.

9.. Sinais e temporização: `scripts/principal.gd` procura inimigos no começo;

inimigos adicionados só depois por uma cena carregada podem exigir conexão em outro lugar.

10.. Arte e cache: não confundir `.godot/` ou arquivos importados com assets

de autoria do projeto.

As observações acima são baseadas em leitura estática dos caminhos citados. Confirme no Godot antes de apresentar uma delas como comportamento visível.

10. Leitura guiada dos códigos

Esta seção acompanha o raciocínio de cada grupo de scripts para ajudar a explicar o código em voz alta. O objetivo é entender responsabilidades e sequência de chamadas, não decorar cada linha.

10.1 Inicialização e carregamento de fases

`scripts/principal.gd`

- `_ready()` prepara o nó principal para coordenar fases.
- `_setup_level()` informa ao autoload qual nó receberá a fase e quais são os

dados de níveis.

- `_on_jogador_dano()` é um callback de dano de inimigo.
- A cena é criada antes das fases; por isso, qualquer busca única de inimigos

no início não necessariamente encontra os inimigos de uma cena que ainda não foi carregada.

`scripts/controladorFases.gd`

- `carregarFase(id)` compõe o caminho `res://fases/<id>/fase.tscn`, carrega a

`PackedScene`, instancia e adiciona ao nó pai.

- A troca de fase também remove a instância anterior quando existe.
- Métodos de consulta/atualização usam o objeto de dados correspondente para

marcar desbloqueio ou conclusão.

• O controlador centraliza a navegação; a cena não deve precisar conhecer os

detalhes internos de todas as outras cenas.

`scripts/dadosFase.gd` e `scripts/fase.gd`

• `dadosFase` é um recurso simples: id, nome, concluída e desbloqueada.

• A raiz da cena de fase contém `idFase`; `fase.gd` usa esse id para buscar o

recurso correspondente.

• Essa separação permite que o controlador e a cena compartilhem o estado sem

guardar todos os dados dentro do script de movimento do jogador.

`scripts/menu.gd` e `fases/0/seletor.gd`

• O menu inicial encaminha o usuário para o seletor ou encerra o jogo.

• O seletor recebe o clique do botão e solicita ao controlador o id escolhido.

• A disponibilidade visual e a permissão real para carregar são conceitos

distintos: o seletor pode mostrar estados, mas o controlador também deve validar o id/desbloqueio se o projeto precisar impor essa regra.

10.2 Jogador legado — `prefab/jogador/jogador.gd`

Uma leitura produtiva é dividir o script em quatro responsabilidades:

1.. Estado e parâmetros: velocidade, gravidade, força de salto e dados de

vida inicializam o jogador.

2.. Movimento: o ciclo de física consulta as ações esquerda/direita, aplica

gravidade e move o corpo; o salto variável e o coyote time tornam o controle mais tolerante.

3.. Plataformas: a entrada `baixo` é usada para descer de uma plataforma

direcional, em vez de tratar a plataforma como chão sólido permanente.

4.. Dano e vida: `perder_vida`, `ganhar_vida`, `rebater_ao_pisar`, `morto` e

`resetar` são métodos que outras cenas/HUD podem chamar para alterar o estado do personagem.

`prefab/jogador/jogador.tscn` fornece os nós físicos e visuais esperados pelo script. O código e a cena formam um par: renomear um nó sem atualizar o caminho usado pelo script pode quebrar a referência `$Node`.

10.3 Jogador moderno — `scrips/player.gd`

O script moderno organiza movimento por estados. Leia o `enum State` antes de ler o processo de física:

Estado	Intenção
`IDLE`	Parado, sem deslocamento horizontal.
`RUN`	Correndo no chão.
`JUMP`	Subindo após o salto.
`FALL`	Descendo sem apoio.
`HURT`	Animação/controle de dano.
`DIE`	Morte e desativação do controle normal.
`CLIMB`	Movimento em escadas.

Sequência para acompanhar no código:

1.. Configuração e `_ready()` localizam sprites, hitbox/hurtbox e

referências; registram grupos de jogador e preparam sinais.

2.. `_physics_process(delta)` lê entradas, calcula o eixo vertical,

aplica gravidade/movimento e escolhe estado/animação.

3.. `_obter_eixo_vertical()` dá compatibilidade entre nomes de ações, como

`up/down` e ações próprias do projeto.

4.. Escada: `entrar_na_escada(escada)` guarda a área ativa; `sair_da_escada`

limpa a referência e encerra a escalada quando necessário.

5.. Dano e vida: os métodos de dano ajustam vida/knockback e estado;

recuperação não deixa ultrapassar a vida máxima.

6.. Reset/morte: métodos específicos restauram vida/posição ou concluem a

sequência de morte.

`actors/player.tscn` é a montagem do personagem: colisão física, sprite, `HitBox`, `HurtBox` e referências para câmera/animações. O arquivo `.gd` não cria sozinho todo o visual.

10.4 Compatibilidade e combate

`scripts/jogador_compat.gd` oferece uma função compartilhada para reconhecer o jogador. A ordem conceitual é: verificar classe/tipo, conferir grupos e aceitar nomes compatíveis. Ela é usada para que objetos não precisem copiar repetidamente as comparações `"jogador"`/`"player"`.

`scrips/hit_box.gd` e `scrips/hurt_box.gd` dividem o combate:

- Hitbox: monitora áreas, filtra autoacerto e contato inválido e encaminha

dados do ataque.

- Hurtbox: oferece o ponto de entrada `take_damage` e emite o sinal

`damage_received`.

- O personagem receptor escolhe o resultado: diminuir vida, aplicar impulso,

entrar em `HURT` ou iniciar `DIE`.

Essa divisão evita acoplar o sprite do atacante ao script de vida do alvo. Se o dano não funcionar, siga a ordem: camada/máscara → monitoramento da área → hitbox reconheceu hurtbox → sinal emitido → callback conectado → mudança de vida → HUD.

10.5 Coletáveis e HUD

- `scrips/item_base.gd`: código comum para iniciar feedback de coleta e

remover o objeto depois da animação.

- `scrips/cherry.gd`: reconhece o jogador e comunica recuperação de vida.
- `scrips/gem.gd`: marca a gema como coletável/diamante e emite o evento

usado pelo contador.

- `prefab/coletaveis/cereja/cereja.gd`: fluxo legado de cereja com feedback

e recuperação.

- `prefab/coletaveis/gema/gema.gd`: fluxo legado de gema com feedback.
- `scrips/vida.gd`: localiza o jogador, escuta eventos e atualiza os ícones

do HUD moderno.

- `prefab/jogador/vida.gd`: alternativa antiga do HUD de vidas.
- `scrips/pontuacao_ui.gd`: consulta gemas/coletáveis e atualiza texto ou

número de pontuação.

Um item precisa de mais do que uma imagem: sua área de colisão precisa detectar o jogador, o callback deve validar o corpo, o efeito/sinal deve ser executado e o item precisa ser removido/ocultado uma única vez.

10.6 Inimigos

- `scrips/opossum.gd`: inicializa vida/direção e percorre uma distância de

patrulha; colisões e sinais de dano trocam o estado entre patrulha, ferido e morto.

- `scrips/frog.gd`: controla o ciclo de salto e deslocamento do sapo, além

das respostas de colisão/dano.

- `scrips/eagle.gd`: atualiza deslocamento e direção da águia voadora e

responde a dano/morte; a cena associa animações e áreas.

- `prefab/inimigos/gamba/gamba.gd`: comportamento antigo de gambá; verifica

contato com o jogador e pode reagir a pisão.

- `prefab/inimigos/sapo/sapo.gd`: sapo antigo, com salto e mudança de

direção.

- `aguia.gd` e `scripts/inimigo.gd`: caminho antigo, distinto da classe

moderna. `scripts/inimigo.gd` é uma base mínima; o comportamento visível fica em `aguia.gd`.

Nos inimigos modernos, a cena `.tscn` define formato/tamanho das áreas, animações e ligação ao script. O script define o que acontece quando essas áreas detectam uma colisão.

10.7 Barreiras, cenário e interações especiais

- `prefab/controle/chao.gd`: trata a queda fora da área segura e solicita a

sequência de morte/reset.

- `prefab/controle/mensagens.gd`: controla a apresentação das mensagens

ligadas a morte/vitória.

- `prefab/controle/final.gd`: caminho de conclusão usado pelo prefab

antigo.

- `scripts/fundo.gd`: atualiza o movimento das camadas de fundo em relação

à câmera, criando parallax.

- `scrips/escada.gd`: converte sinais `body_entered/body_exited` em chamadas

aos métodos de escada do jogador.

- `scrips/moving_plataform.gd`: move uma plataforma entre referências,

aguarda e inverte/repete o ciclo.

- `scrips/porta_saida.gd`: valida proximidade/entrada, apresenta interação,

anima transição e exibe vitória.

- `scrips/area_secreta_01.gd`: altera de forma animada o limite de câmera

durante entrada/saída da região.

- `scrips/area_secreta_02.gd`: anima a transparência de um `Polygon2D`

associado à região secreta.

- `fases/1/fase.tscn.gd`: possui propriedades exportadas para id e início

do jogador, mas não foi encontrado anexado a uma cena analisada.

10.8 Cenas duplicadas e conexões serializadas

O projeto também contém `Fase05.tscn` na raiz além de `fases/5/fase.tscn`. As duas cenas declaram o mesmo UID; o controlador constrói o caminho `fases/5/fase.tscn`, então esse é o arquivo usado pelo fluxo normal descrito. Antes de editar uma fase, confirme qual caminho é carregado.

Uma conexão de sinal salva dentro de `.tscn` é diferente de uma conexão feita por código em `_ready()`. Se ambos tentarem ligar o mesmo evento, verifique duplicação; se o método de destino foi renomeado, a conexão serializada pode estar obsoleta. A conexão da águia em `actors/eagle.tscn` é um exemplo útil para demonstrar essa diferença.

11. Mini roteiro de apresentação (5–7 minutos)

1.. Objetivo e gênero: jogo de plataforma 2D; percorrer fases, evitar perigos e coletar itens.

2.. Tecnologia: Godot 4.7.1, GDScript, cenas reutilizáveis, TileMap, sprites, colisões e sinais.

3.. Fluxo: menu → seletor (id 0) → fase 1–5, coordenado pelo autoload.

4.. Arquitetura: jogador legado nas fases 1/2/4 e moderno nas fases 3/5.

5.. Jogabilidade: movimento/pulo, dano e inimigos, itens/vida, escadas e saída.

6.. Demonstração: escolher uma fase, coletar uma gema, tomar dano e mostrar uma cena reutilizável da árvore.

7.. Aprendizado técnico: explique um sinal (hitbox → hurtbox → jogador/HUD) e um ponto de evolução do projeto (compatibilidade entre jogadores).

Perguntas para praticar em grupo

- Qual é a diferença entre cena e instância?
- Por que o `controladorFases` é autoload?
- O que acontece depois de pressionar Jogar?
- Como a hitbox aplica dano sem conter toda a lógica do inimigo?
- Por que uma gema pertence à camada/grupo de coletáveis?
- Quais fases usam cada jogador e o que muda entre eles?
- O que um teste headless confirma e o que ainda precisa de teste manual?
- Qual recurso do Godot tornou mais fácil montar os níveis?

12. Glossário curto para a apresentação

- Coyote time: pequena tolerância de tempo para saltar depois de sair de uma plataforma.
- Knockback: impulso aplicado ao personagem ao sofrer um golpe.
- Hitbox: área que procura um alvo para atacar.
- Hurtbox: área que representa a região vulnerável de um personagem.
- Parallax: movimento em velocidades diferentes de camadas do fundo para criar profundidade.
- Instanciar: criar em execução ou inserir em outra cena uma cópia de uma cena reutilizável.
- Headless: executar o Godot sem abrir a interface gráfica; útil para

validar carregamento e erros, mas não para sentir o controle.

- Cena procedural? Nesta cópia, as fases jogáveis são principalmente cenas

serializadas no editor. Isso não deve ser confundido com o outro projeto, `sunnyland_5_fases`, que gera mapas a partir de texto.

13. Arquivos para abrir primeiro

- Configuração do projeto
- Cena principal
- Controlador de fases
- Entrada e menu · Menu
- Seletor · Script do seletor
- Jogador legado · Cena do jogador legado
- Jogador moderno · Cena do jogador moderno
- Fase 3 · Fase 4 · Fase 5